

Lab 6: Copy-on-Write (COW) Fork 实验报告

李睿阳 524120910059

1. Subtasks 实现思路与关键代码

1.1 物理页引用计数 (Reference Counting)

实现思路：

由于 COW 允许多个进程共享同一块物理内存，我们需要记录每个物理页被引用的次数。只有当引用计数降为 0 时，该物理页才会被真正释放回空闲链表。我们在 `kernel/kalloc.c` 中增加一个全局数组 `p_ref.count`，以物理页地址除以 `PGSIZE` 作为索引。

修改文件：

- [kernel/kalloc.c](#)
- [kernel/defs.h](#)

关键代码：

```

// kernel/kalloc.c
struct {
    struct spinlock lock;
    int count[PHYSTOP / PGSIZE];
} p_ref;

void kfree(void *pa) {
    // ... 检查地址合法性 ...
    acquire(&p_ref.lock);
    if (p_ref.count[(uint64)pa / PGSIZE] > 1) {
        p_ref.count[(uint64)pa / PGSIZE]--;
        release(&p_ref.lock);
        return;
    }
    p_ref.count[(uint64)pa / PGSIZE] = 0;
    release(&p_ref.lock);
    // 执行真正的内存释放逻辑...
}

void kref_inc(uint64 pa) {
    acquire(&p_ref.lock);
    p_ref.count[pa / PGSIZE]++;
    release(&p_ref.lock);
}

```

1.2 定义 COW 标志位

实现思路：

利用 RISC-V 页表项中的保留位（RSW）中的一位来标记该页面是否为 COW 页面。

修改文件：

- [kernel/riscv.h](#)

关键代码：

```

// kernel/riscv.h
#define PTE_COW (1L << 8) // 使用第 8 位作为 COW 标记

```

1.3 修改 `uvmcopy()`

实现思路：

在 `fork()` 调用 `uvmcopy()` 复制父进程页表到子进程时，不再为子进程分配新的物理内存。而是将父进程的物理页映射到子进程，并清除原本可写页面的 `PTE_W` 标志位，同时添加 `PTE_COW` 标志位。最后增加该物理页的引用计数。

修改文件：

- [kernel/vm.c](#)

关键代码：

```
// kernel/vm.c
if (*pte & PTE_W) {
    *pte &= ~PTE_W;
    *pte |= PTE_COW;
}
flags = PTE_FLAGS(*pte);
if(mappages(new, i, PGSIZE, pa, flags) != 0) goto err;
kref_inc(pa); // 物理页引用加 1
```

1.4 实现 COW 页错误处理程序

实现思路：

在 `kernel/vm.c` 中实现 `cow_handler`。当 CPU 触发“写异常”(scause 15) 时，通过 `usertrap()` 调用该处理程序。它会检查异常地址是否对应一个 COW 页面。若是，则分配一个新物理页，复制原页内容，更新页表指向新物理页并恢复 `PTE_W`。如果原页面引用计数已经为 1，则可以直接恢复写权限而无需分配新页。

修改文件：

- [kernel/vm.c](#)
- [kernel/trap.c](#)

关键代码：

```

// kernel/vm.c
int cow_handler(pagetable_t pagetable, uint64 va) {
    // ... 查找 pte 和获取 pa ...
    if(kref_get(pa) == 1) { // 优化：唯一引用直接转为可写
        *pte &= ~PTE_COW;
        *pte |= PTE_W;
    } else {
        char *mem = kalloc();
        if(mem == 0) return -1;
        memmove(mem, (char*)pa, PGSIZE);
        uint flags = PTE_FLAGS(*pte);
        flags |= PTE_W;
        flags &= ~PTE_COW;
        *pte = PA2PTE(mem) | flags; // 更新页表映射
        kfree((void*)pa); // 减少旧物理页计数
    }
}

// kernel/trap.c (usertrap 函数中)
} else if(r_scause() == 15){ // Store page fault
    if(cow_handler(p->pagetable, r_stval()) < 0)
        setkilled(p);
}

```

1.5 修改 copyout()

实现思路：

内核在通过 copyout 向用户空间写入数据时，是通过软件 walk 页表来获取地址的，不会触发硬件页异常。因此需要手动检查目标页是否为 COW 页面，并在写入前调用 cow_handler 分配新的物理页。

修改文件：

- [kernel/vm.c](#)

关键代码：

```
// kernel/vm.c (copyout 函数循环内)
if(!(*pte & PTE_W)){
    if(*pte & PTE_COW){
        if(cow_handler(pagetable, va0) < 0) return -1;
        pte = walk(pagetable, va0, 0); // 重新获取更新后的 pte
    } else return -1;
}
```

2. 调试中的要点

1. **并发安全 (Locking)**: 物理页引用计数数组 `p_ref.count` 是全局共享资源。在多核环境下, 必须使用自旋锁 (Spinlock) 保护所有的读、写和修改操作, 否则会导致计数不一致, 进而出现内存泄漏或过早释放物理页的情况。
2. **kfree 的行为改变**: 传统的 `kfree` 直接将物理页放回空闲链表。在 COW 环境下, `kfree` 仅仅是“尝试释放”。必须先减少计数, 只有计数归零时才真正释放。
3. **引用计数初始化**: 物理页在系统启动、`kalloc` 调用以及在 `uvmcopy` 共享时, 必须准确地管理计数的增加与减少。尤其是 `kalloc` 成功后, 计数值必须初始化为 1。
4. **页表更新同步**: 在 `cow_handler` 修改页表项 (PTE) 后, 由于更改了物理映射, 理论上需要刷新 TLB (RISC-V 中使用 `sfence.vma`), 但在 xv6 的环境下, 从内核返回用户态时会自动刷新, 因此在此处显式刷新不是必须的, 但对于性能或在其他更复杂的 OS 中可能很重要。
5. **内存不足处理**: 如果在 `cow_handler` 中 `kalloc` 失败 (无空闲物理页), 必须捕获此错误并终止对应进程 (`setkilled`), 否则会导致内核 panic 或不确定的行为。

3. 测试结果 (make grade)

已经通过 `cowtest` 和 `usertests -q` 所有测试。

```
== Test    simple ==
  simple: OK
== Test    three ==
  three: OK
== Test    file ==
  file: OK
== Test usertests ==
  usertests: copyin: OK
  usertests: copyout: OK
  usertests: all tests: OK
```